

Summer 2026 Data C100/C200 Final Reference Sheet

Canonical Tabular Manipulations

Manipulation

Select `column(s)`.

Filter to rows with specific `value(s)` in `column(s)`.

Create new `column`.

Combine `columns` element-wise using addition, subtraction, or another `operation`.

Keep only first `n` row(s).

Keep only `nth` row(s).

Aggregate values in `column(s)` (e.g., using `sum` or `average`).

Count number of rows.

Count number of unique values in a `column`.

Sort or order by value of `column(s)` in ascending or descending order.

Left/right/inner/outer join two `datasets` based on matching value of `column(s)`.

For each unique value of a `column` (or combination of `columns`), do `{...}`.

Visualization

Matplotlib: `x` and `y` are sequences of values. `import matplotlib.pyplot as plt`

Function	Description
<code>plt.plot(x, y)</code>	Creates a line plot of <code>x</code> against <code>y</code>
<code>plt.scatter(x, y)</code>	Creates a scatter plot of <code>x</code> against <code>y</code>
<code>plt.hist(x, bins=None)</code>	Creates a histogram of <code>x</code> ; <code>bins</code> can be an integer or a sequence
<code>plt.bar(x, height)</code>	Creates a bar plot of categories <code>x</code> and corresponding heights <code>height</code>

Seaborn: `x` and `y` are column names in a DataFrame `data`. `import seaborn as sns`

Function	Description
<code>sns.countplot(data=None, x=None)</code>	Create a barplot of value counts of variable <code>x</code> from <code>data</code>
<code>sns.histplot(data=None, x=None, stat='count', kde=False)</code> <code>sns.displot(data=None, x=None, kind='hist', rug=False)</code>	Creates a histogram of <code>x</code> from <code>data</code> , where bin statistics <code>stat</code> is one of <code>'count'</code> , <code>'frequency'</code> , <code>'probability'</code> , <code>'percent'</code> , and <code>'density'</code> ; optionally overlay a kernel density estimator. <code>displot</code> is similar but can optionally overlay a rug plot and/or a KDE plot
<code>sns.rugplot(data=None, x=None)</code>	Adds a rug plot on the x-axis of variable <code>x</code> from <code>data</code>
<code>sns.boxplot(data=None, x=None, y=None)</code> <code>sns.violinplot(data=None, x=None, y=None)</code>	Create a boxplot of a numeric feature (e.g., <code>y</code>), optionally factoring by a category (e.g., <code>x</code>), from <code>data</code> . <code>violinplot</code> is similar but also draws a kernel density estimator of the numeric feature
<code>sns.scatterplot(data=None, x=None, y=None)</code>	Create a scatterplot of <code>x</code> versus <code>y</code> from <code>data</code>
<code>sns.lmplot(data=None, x=None, y=None, fit_reg=True)</code>	Create a scatterplot of <code>x</code> versus <code>y</code> from <code>data</code> , and by default overlay a least-squares regression line
<code>sns.jointplot(data=None, x=None, y=None, kind='scatter')</code>	Combine a bivariate scatterplot of <code>x</code> versus <code>y</code> from <code>data</code> , with univariate density plots of each variable overlaid on the axes; <code>kind</code> determines the visualization type for the distribution plot, can be <code>scatter</code> , <code>kde</code> or <code>hist</code>

Regular Expressions

Operator	Description	Operator	Description
.	Matches any character except \n	*	Matches preceding character/group zero or more times
\	Escapes metacharacters	?	Matches preceding character/group zero or one times
	Matches expression on either side of expression; has lowest priority of any operator	+	Matches preceding character/group one or more times
\d, \w, \s	Predefined character group of digits (0-9), alphanumerics (a-z, A-Z, 0-9, and underscore), or whitespace, respectively	^, \$	Matches the beginning and end of the line, respectively
\D, \W, \S	Inverse sets of \d, \w, \s, respectively	()	Capturing group used to create a sub-expression
{m}	Matches preceding character/group exactly m times	[]	Character class used to match any of the specified characters or range (e.g. [abcde] is equivalent to [a-e])
{m, n}	Matches preceding character/group at least m times and at most n times. If either m or n are omitted, set lower/upper bounds to 0 and ∞, respectively	[^]	Invert character class; e.g. [^a-c] matches all characters except a, b, c

Modified lecture example for capture groups:

```
import re
lines = '169.237.46.168 - - [26/Jan/2014:10:47:58 -0800] "GET ... HTTP/1.1"'
re.findall(r'\[(\d+\./(\w+)\./\d+:\d+:\d+:\d+ .+)\]', lines) # returns ['Jan']
```

Function	Description
re.match(pattern, string)	Returns a match if zero or more characters at beginning of string matches pattern, else None
re.search(pattern, string)	Returns a match if zero or more characters anywhere in string matches pattern, else None
re.findall(pattern, string)	Returns a list of all non-overlapping matches of pattern in string (if none, returns empty list)
re.sub(pattern, repl, string)	Returns string after replacing all occurrences of pattern with repl

Modeling

Concept	Formula	Concept	Formula
Variance, σ_x^2	$\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$	Correlation r	$r = \frac{1}{n} \sum_{i=1}^n \frac{x_i - \bar{x}}{\sigma_x} \frac{y_i - \bar{y}}{\sigma_y}$
L_1 loss	$L_1(y, \hat{y}) = y - \hat{y} $	Linear regression estimate of y	$\hat{y} = \theta_0 + \theta_1 x$
L_2 loss	$L_2(y, \hat{y}) = (y - \hat{y})^2$	Least squares linear regression	$\hat{\theta}_0 = \bar{y} - \hat{\theta}_1 \bar{x} \quad \hat{\theta}_1 = r \frac{\sigma_y}{\sigma_x}$
Empirical risk with loss L			
	$R(\theta) = \frac{1}{n} \sum_{i=1}^n L(y_i, \hat{y}_i)$		

Multiple Linear Regression Formulas

Concept	Formula	Concept	Formula
Mean squared error	$R(\theta) = \frac{1}{n} Y - \mathbb{X}\theta _2^2$	Normal equation	$\mathbb{X}^T \mathbb{X} \hat{\theta} = \mathbb{X}^T Y$
Least squares estimate, if $\mathbb{X}$ is full rank	$\hat{\theta} = (\mathbb{X}^T \mathbb{X})^{-1} \mathbb{X}^T Y$	Multiple R^2 (coefficient of determination)	$R^2 = \frac{\text{variance of fitted values}}{\text{variance of } y}$

Concept	Formula	Concept	Formula
Ridge Regression L2 Regularization	$\frac{1}{n} \ \mathbb{Y} - \mathbb{X}\theta\ _2^2 + \lambda \ \theta\ _2^2$	Squared L2 Norm of $\theta \in \mathbb{R}^d$	$\ \theta\ _2^2 = \sum_{j=1}^d \theta_j^2$
Ridge regression estimate (closed form)	$\hat{\theta}_{\text{ridge}} = (\mathbb{X}^T \mathbb{X} + n\lambda I)^{-1} \mathbb{X}^T \mathbb{Y}$	L1 Norm of $\theta \in \mathbb{R}^d$	$\ \theta\ _1 = \sum_{j=1}^d \theta_j $
LASSO Regression L1 Regularization	$\frac{1}{n} \ \mathbb{Y} - \mathbb{X}\theta\ _2^2 + \lambda \ \theta\ _1$		

Scikit-Learn

Package: `sklearn.linear_model`

Linear Regression	Logistic Regression	Function(s)	Description
✓	-	<code>LinearRegression(fit_intercept=True)</code>	Returns an ordinary least squares Linear Regression model.
-	✓	<code>LogisticRegression(fit_intercept=True, penalty='l2', C=1.0)</code>	Returns an ordinary least squares Linear Regression model. Hyperparameter C is inverse of regularization parameter, C = 1/λ.
✓	-	<code>LassoCV()</code> , <code>RidgeCV()</code>	Returns a Lasso (L1 Regularization) or Ridge (L2 regularization) linear model, respectively, and picks the best model by cross validation.
✓	✓	<code>model.fit(X, y)</code>	Fits the scikit-learn <code>model</code> to the provided <code>x</code> and <code>y</code> .
✓	✓	<code>model.predict(X)</code>	Returns predictions for the <code>x</code> passed in according to the fitted <code>model</code> .
✓	✓	<code>model.predict_proba(X)</code>	Returns predicted probabilities for <code>x</code> according to the fitted <code>model</code> . If binary classes, will return probabilities for both class 0 and 1.
✓	✓	<code>model.coef_</code>	Estimated coefficients for the linear model, excluding the intercept.
✓	✓	<code>model.intercept_</code>	Bias/intercept term of the linear model. Set to 0.0 if <code>fit_intercept=False</code> .

Package: `sklearn.model_selection`

Function	Description
<code>train_test_split(*arrays, test_size=0.2)</code>	Returns two random subsets of each array passed in, with 0.8 of the array in the first subset and 0.2 in the second subset.

Probability

Let X have a discrete probability distribution $P(X = x)$. X has expectation $\mathbb{E}[X] = \sum_x xP(X = x)$ over all possible values x , variance $\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2]$, and standard deviation $\text{SD}(X) = \sqrt{\text{Var}(X)}$.

Notes	Property of Expectation	Property of Variance
X is a random variable.	$\mathbb{E}[X] = \sum_x xP(X = x)$	$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = E[X^2] - (E[X])^2$
X is a random variable, $a, b \in \mathbb{R}$ are scalars.	$\mathbb{E}[aX + b] = a\mathbb{E}[X] + b$	$\text{Var}(aX + b) = a^2 \text{Var}(X)$
X, Y are random variables.	$\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$	$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2\text{Cov}(X, Y)$
X is a Bernoulli random variable that takes the value 1 with probability p , and 0 otherwise.	$\mathbb{E}[X] = p$	$\text{Var}(X) = p(1 - p)$

Parameter Estimation and Gradient Descent Update Rule

Parameter Estimation

Suppose for each individual with fixed input x , we observe a random response $Y = g(x) + \epsilon$, where g is the true relationship and ϵ is random noise with zero mean and variance σ^2 .

For a new individual with fixed input x , define our random prediction $\hat{Y}(x)$ based on a model fit to our observed sample $(\mathbb{X}, \mathbb{Y})$. The model risk is the mean squared prediction error between Y and $\hat{Y}(x)$: $\mathbb{E}[(Y - \hat{Y}(x))^2] = \sigma^2 + \left(\mathbb{E}[\hat{Y}(x)] - g(x)\right)^2 + \text{Var}(\hat{Y}(x))$.

Suppose that input x has p features and the true relationship g is linear with parameter $\theta \in \mathbb{R}^{p+1}$. Then $Y = f(x) = \theta_0 + \sum_{j=1}^p \theta_j x_j + \epsilon$ and $\hat{Y} = \hat{f}(x)$ for an estimate $\hat{\theta}$ fit to the observed sample $(\mathbb{X}, \mathbb{Y})$.

Gradient Descent

For a learning rate α , the gradient update step is:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla_{\theta} L(\theta^{(t)})$$

where $\nabla_{\theta} L(\theta^{(t)})$ is the partial derivative/gradient of L with respect to θ , evaluated at $\theta^{(t)}$.

Logistic Regression and Classification

Logistic Regression Model: For input feature vector x , $\hat{P}_{\theta}(Y = 1|x) = \sigma(x^T \theta)$, where $\sigma(z) = 1/(1 + e^{-z})$. For a single datapoint, define cross-entropy loss as $-[y \log(p) + (1 - y) \log(1 - p)]$, where p is the probability that the response is 1.

An ROC curve has the false positive rate (FPR) on the x-axis and true positive rate (TPR) on the y-axis.

Classification Performance

Suppose you predict n datapoints.

Metric	Formula
Accuracy	$\frac{TP+TN}{n}$
Precision	$\frac{TP}{TP+FP}$
Recall, True Positive Rate (TPR)	$\frac{TP}{TP+FN}$
False Positive Rate (FPR)	$\frac{FP}{FP+TN}$
F1 Score	$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$

Clustering

A datapoint's **silhouette score** S is defined as $S = (B - A) / \max(A, B)$, where A is the mean distance to other points in its cluster, and B is the mean distance to points in its closest cluster.